

COGNOME:

NOME:

MATRICOLA:

## Appello di Algoritmi e Strutture Dati (9CFU), 9 Febbraio 2026

Parte I - Usare voto prova parziale - SI( ) NO( )

**Esercizio 1 [3 punti]**Quale delle seguenti sequenze di funzioni è tale che ogni funzione è  $O$  della successiva?

(A)  $\log n^{2025}$ ,  $\log n^{2026}$ ,  $\sum_{n=1}^{2026} \log n$ ,  $2^{\log n}$ ,  $\binom{n}{2}$

~~(B)~~  $\sum_{i=0}^{\log n} 2^i$ ,  $\sum_{i=1}^n \log i$ ,  $2^{\sqrt{n}}$ ,  $2^n$ ,  $\sum_{i=0}^n 2^i$

(C) Nessuna delle altre

(D)  $\log n^{2026}$ ,  $\log n^{2025}$ ,  $\frac{n^{1.001}}{(\log n)^{2026}}$ ,  $n$ ,  $n^2$

**Esercizio 2 [4 punti]**Si consideri il seguente algoritmo, in cui l'input è una potenza di 2. Qual è il numero di chiamate alla funzione  $f$ ?**Algorithm** ALGO( $n$ )1: if  $n = 1$  then2:  $f()$ 3: if  $n > 1$  then4: ALGO( $n/2$ )5: for  $i = 1, \dots, \log n$  do  $i \rightarrow \log n$ 6: for  $j = i, \dots, n$  do  $j \rightarrow n$ 7:  $f()$ Quando  $i=1$   $j: 1 \rightarrow n$  (n passi) $i=2$   $j: 2 \rightarrow n$  (n-1 passi) $\vdots$   
 $i: \log n$   $j: \log n \rightarrow n$  (n - log n + 1 passi)Somma =  $\frac{n + (n-1) + \dots + (n - (\log n - 1))}{\log n \text{ termini}}$ 

~~(A)~~  $\Theta(n \log n)$

(B)  $\Theta(n^2)$

(C)  $\Theta(\log n)$

(D) Nessuna delle altre

$$T(n) = T(n/2) + \Theta(n \log n)$$

$$a=1 \quad b=2 \quad n^{\log a} = n^0 = 1 < n \log n$$

$$T(n) = \Theta(n \log n)$$

**Esercizio 3 [3 punti]**

Quale delle seguenti affermazioni è vera?

~~(A)~~ Supponiamo di utilizzare MERGE-SORT per ordinare un array  $A$  di  $n$  interi. Allora qualsiasi elemento dell'array  $A$  è confrontato con  $O(\log n)$  elementi di  $A$ .(B) Dato un array ordinato  $A$  di  $n$  interi, vogliamo ordinare l'array formato dai quadrati degli elementi di  $A$ . Per esempio, se  $A = \langle -4, -1, 0, 2, 5 \rangle$ , allora l'array cercato è  $\langle 0, 1, 4, 16, 25 \rangle$ . L'algoritmo più efficiente per risolvere questo problema ha tempo di esecuzione  $\Theta(n \log n)$ .(C) La ricorrenza per la complessità temporale dell'algoritmo di Karatsuba è  $T(n) = 4T(n/2) + \Theta(n)$ .

(D) Nessuna delle altre.

Parte II

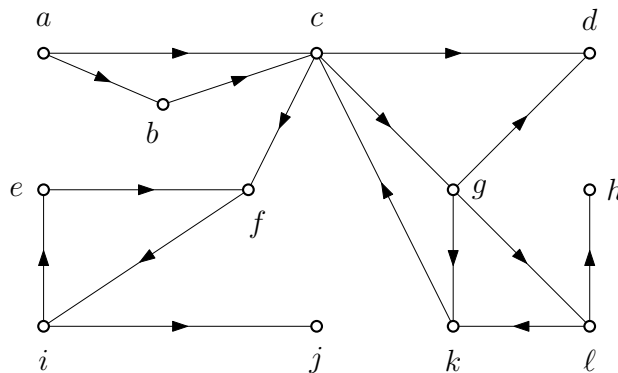
**Esercizio 4 [3 punti]**Si consideri il max-heap  $A$  rappresentato dall'array  $\langle 55, 30, 50, 25, 15, 5, 10, 20 \rangle$ . La rappresentazione del max-heap  $A$  tramite array dopo chiamate, nell'ordine, a MAX-HEAP-INSERT( $A, 35$ ), HEAP-EXTRACT-MAX( $A$ ),

MAX-HEAP-INSERT( $A, 21$ ), e HEAP-EXTRACT-MAX( $A$ ) è: ~~(A)~~  $\langle 35, 30, 25, 21, 15, 5, 10, 20 \rangle$   
 (B)  $\langle 35, 30, 25, 20, 15, 5, 10, 21 \rangle$  (C)  $\langle 30, 25, 50, 35, 15, 5, 10, 20 \rangle$  (D) Nessuna delle altre

### Esercizio 5 [4 punti]

Si consideri il grafo diretto  $G$  in figura. Si effettui una visita in profondità di  $G$  a partire da  $a$  in cui i vertici sono ordinati alfabeticamente, così come ciascuna lista di adiacenza. Quale delle seguenti affermazioni è vera?

- (A) Nessuna delle altre.  
 (B)  $(\ell, k)$  è l'unico arco trasversale.  
 (C) Il numero minimo di archi che è necessario rimuovere da  $G$  in modo che il grafo risultante ammetta un ordinamento topologico è 4.  ~~$\rightarrow$  no, 2,  $(e, f), (h, c)$~~   
~~(D)~~ L'insieme degli archi all'indietro è  $\{(e, f), (k, c)\}$ .



### Esercizio 6 [3 punti]

Quale delle seguenti affermazioni è vera?

- (A) Nessuna delle altre.  
 (B) Sia  $G = (V, E)$  un grafo non diretto e pesato arbitrario, con funzione peso positiva, e sia  $T$  un albero di connessione minimo di  $G$ . Dati due vertici qualsiasi  $s, t \in V$ , il cammino tra  $s$  e  $t$  in  $T$  è un cammino minimo di  $G$ .  
 (C) Se in un ordinamento topologico di un grafo diretto  $G$  il vertice  $u$  precede il vertice  $v$ , allora esiste un cammino in  $G$  da  $u$  a  $v$ .  
~~(D)~~ Se un grafo diretto aciclico  $G = (V, E)$  ammette un unico ordinamento topologico, allora per ogni coppia di vertici distinti  $u, v \in V$  esiste un cammino in  $G$  da  $u$  a  $v$  oppure da  $v$  a  $u$ .

---

SBARRAMENTO - 12 punti sugli esercizi precedenti

---

**Esercizio 7 [8 punti]** Si consideri l'algoritmo di Bellman-Ford. Se ne descriva brevemente l'idea, e si forniscano pseudo codice e tempo di esecuzione nel caso peggiore.

**Esercizio 8 [4 punti]** Vogliamo trovare cammini minimi da sorgente singola per un grafo diretto  $G = (V, E)$  con funzione peso  $w: E \rightarrow \mathbb{R}$  positiva (i.e.,  $w(e) > 0$  per ogni  $e \in E$ ).

1. Se  $m = \Theta(n)$ , quale algoritmo tra BELLMAN-FORD e DIJKSTRA è più efficiente?
2. Supponiamo ora che i pesi siano interi positivi, vale a dire  $w: E \rightarrow \{1, 2, \dots, d\}$  per un certo  $d \in \mathbb{N}$ . Mostrare come sia possibile applicare BFS ad un certo grafo  $G'$  costruito a partire da  $G$  in modo da risolvere il nostro problema in  $O(n + m)$ .

### Esercizio 1 [3 punti]

Quale delle seguenti sequenze di funzioni è tale che ogni funzione è  $O$  della successiva?

(A)  $\log n^{2025}$ ,  $\log n^{2026}$ ,  $\sum_{n=1}^{2026} \log n$ ,  $2^{\log n}$ ,  $\binom{n}{2}$

(B)  $\sum_{i=0}^{\log n} 2^i$ ,  $\sum_{i=1}^n \log i$ ,  $2^{\sqrt{n}}$ ,  $2^n$ ,  $\sum_{i=0}^n 2^i$

(C) Nessuna delle altre

(D)  $\log n^{2026}$ ,  $\log n^{2025}$ ,  $\frac{n^{1.001}}{(\log n)^{2026}}$ ,  $n$ ,  $n^2$

A.  $\log n^{2025} \leq \log n^{2026} > \sum_{i=0}^{2026} \log n$

•  $\log n^{2025} = 2025 \log n = O(\log n)$

•  $\log n^{2026}$

•  $\sum_{i=0}^{2026} \log n \approx \log 2027$

B.  $\sum_{i=0}^{\log n} 2^i \leq \sum_{i=1}^n 2^i \leq 2^{\sqrt{n}} \leq 2^n \leq \sum_{i=0}^n 2^i$

•  $\sum_{i=0}^{\log n} 2^i \approx 2^{\log(n+1)} = n+1$

•  $\sum_{i=1}^n \log i = \log(1 \cdot 2 \cdot \dots \cdot n) = \log(n!) \approx n \log n$

•  $2^{\sqrt{n}} = 2^{n^{1/2}}$

•  $\sum_{i=0}^n 2^i \approx 2^{n+1} = 2^n \cdot 2$

D.  $\log n^{2026} > \log n^{2025}$

•  $\log n^{2026} = 2026 \log n$

•  $\log n^{2025} = 2025 \log n$

**Esercizio 7 [8 punti]** Si consideri l'algoritmo di Bellman-Ford. Se ne descriva brevemente l'idea, e si forniscano pseudo codice e tempo di esecuzione nel caso peggiore.

- Bellman-Ford è un algoritmo che risolve il problema dei cammini minimi da sorgente singola su un grafo singolo e diretto  $G=(V,E)$ , anche in presenza di pesi negativi.
- L'idea si basa sulla tecnica del rilassamento degli archi, combinata con la programmazione dinamica:
  1. si inizializzano le distanze dalla sorgente a tutti gli altri nodi a  $+\infty$  e 0 per la sorgente stessa
  2. si esegue un ciclo principale che effettua il rilassamento di tutti gli archi del grafo per  $|V|-1$  volte (perché il cammino minimo più lungo possibile senza cicli in un grafo contiene al massimo  $|V|-1$  archi)
  3. al termine dei cicli, si effettua un ultimo controllo su ciascun arco, se è ancora possibile ridurre la distanza di un nodo, significa che nel grafo è presente un ciclo di peso negativo raggiungibile dalla sorgente, in tal caso l'algoritmo segnala l'anomalia e restituisce false.

INITIALIZE\_SINGLE-SOURCE( $G, s$ )

Per ogni vertice  $v \in V$  do

$v.d = +\infty$

$v.\pi = NIL$

$s.d = 0$

RELAX( $G, w, s$ )

Se  $v.d > u.d + w(u, v)$  allora

$v.d = u.d + w(u, v)$

$v.\pi = u$

BELLMAN-FORD( $G, w, s$ )

INITIALIZE\_SINGLE-SOURCE( $G, s$ )

→  $O(|V|)$

for  $i = 1$  to  $|V|-1$  do

Per ogni arco  $(u, v) \in E$  do

RELAX( $u, v, w$ )

$|V|-1 \cdot O(|E|)$   
=  $O(|V| \cdot |E|)$

Per ogni arco  $(u, v) \in E$  do

Se  $v.d > u.d + w(u, v)$  allora

restituisce FALSE

$O(|E|)$

restituisce TRUE

Costo Totale :  $O(|V| \cdot |E|)$  nel caso peggiore

**Esercizio 8 [4 punti]** Vogliamo trovare cammini minimi da sorgente singola per un grafo diretto  $G = (V, E)$  con funzione peso  $w: E \rightarrow \mathbb{R}$  positiva (i.e.,  $w(e) > 0$  per ogni  $e \in E$ ).

1. Se  $m = \Theta(n)$ , quale algoritmo tra BELLMAN-FORD e DIJKSTRA è più efficiente?
2. Supponiamo ora che i pesi siano interi positivi, vale a dire  $w: E \rightarrow \{1, 2, \dots, d\}$  per un certo  $d \in \mathbb{N}$ . Mostrare come sia possibile applicare BFS ad un certo grafo  $G'$  costruito a partire da  $G$  in modo da risolvere il nostro problema in  $O(n + m)$ .

1. Bellman-Ford nel caso peggiore costa  $O(n \cdot m) \xrightarrow{m = \Theta(n)} O(n^2)$

Dijkstra (con heap) nel caso peggiore costa  $O((m+n) \log n) \xrightarrow{m = \Theta(n)} O((n+n) \log n) = O(n \log n)$

Di conseguenza Dijkstra è più efficiente

2. BFS calcola i cammini minimi solo se i pesi degli archi sono unitari, qui i pesi sono interi generici da 1 a  $d$ . L'idea è smontare ogni arco di peso  $k$  in una catena di  $k$  archi con peso 1, inserendo dei nodi intermedi.

La Costruzione di  $G'$ : Dato un grafo originale  $G=(V,E)$  costruiamo un nuovo grafo  $G'=(V',E')$  non pesato nel modo seguente:

1. Tutti i nodi originali vengono mantenuti in  $V'$ .

2. Per ogni arco  $(u,v) \in E$  che possiede un peso  $w(u,v)=k$ :

se  $k=1$ : inseriamo l'arco direttamente in  $G'$

se  $k>1$ : inseriamo  $k-1$  nuovi nodi  $z_1, z_2, \dots, z_{k-1}$  di supporto, e

Collegiamo  $u$  a  $v$  mediante una catena di  $k$  archi

di peso 1, es:  $u \rightarrow z_1 \rightarrow z_2 \rightarrow \dots \rightarrow z_{k-1} \rightarrow v$